

Universidad Sergio Arboleda

Programa de Ciencias de la Computación e Inteligencia Artificial

Proyecto de curso

Lenguaje de Dominio Específico para Ciencia de Datos y Visualización

Lenguajes de Programación y Transducción

Propósito general: diseñar e implementar, con ANTLR4 y Python, un lenguaje de dominio específico que permita expresar flujos reproducibles de carga, preparación, análisis y visualización de datos.

Docente: Joaquín F. Sánchez

Semestre 2026-2

Índice

1. Descripción general	2
2. Alcance funcional del DSL	2
3. Enfoque de diseño	3
4. Arquitectura esperada	4
5. Organización del proyecto por fases	5
6. Criterios de evaluación	7
7. Requisitos técnicos	7
8. Caso de estudio final recomendado	7
9. Resultado esperado del curso	8

1. Descripción general

Un lenguaje de dominio específico (DSL, por sus siglas en inglés) es un lenguaje diseñado para resolver problemas dentro de un campo delimitado. En este proyecto, el dominio seleccionado es la **ciencia de datos y la visualización**, por lo que el lenguaje deberá permitir que un usuario describa, con una sintaxis sencilla y declarativa, las etapas principales de un flujo de análisis de datos.

El proyecto conserva el uso de **ANTLR4** como herramienta para la construcción del analizador léxico y sintáctico, y utiliza **Python** como lenguaje de implementación. La ejecución semántica se realizará mediante el patrón *Visitor* generado por ANTLR. Como apoyo para el procesamiento y la visualización podrán emplearse bibliotecas como *pandas*, *NumPy* y *Matplotlib*.

Objetivo del proyecto

Diseñar e implementar un DSL funcional y reproducible que permita cargar conjuntos de datos, seleccionar y transformar variables, filtrar registros, calcular estadísticas descriptivas y producir visualizaciones, manteniendo una separación clara entre la sintaxis del lenguaje, su semántica y la ejecución sobre bibliotecas de ciencia de datos.

2. Alcance funcional del DSL

El lenguaje deberá cubrir las operaciones esenciales de un flujo reproducible de ciencia de datos. La siguiente tabla establece el alcance mínimo esperado. Las instrucciones, palabras reservadas y detalles sintácticos deberán ser definidos y justificados por cada equipo.

Componente	Capacidades del DSL	Funcionalidad mínima esperada
Carga y almacenamiento	Lectura y escritura de conjuntos de datos.	Cargar archivos CSV; asociar los datos con un identificador; configurar encabezados, separadores y rutas; guardar resultados en CSV. La lectura de JSON o Excel podrá incorporarse como ampliación.
Selección y preparación	Operaciones para organizar, depurar y preparar variables y registros.	Seleccionar columnas; filtrar filas mediante expresiones booleanas; ordenar datos; renombrar variables; eliminar duplicados; tratar valores faltantes; convertir tipos; crear columnas calculadas.
Expresiones y operadores	Construcción de expresiones utilizadas en filtros, transformaciones y cálculos.	Operadores aritméticos de suma, resta, multiplicación, división, módulo y potencia; operadores relacionales; operadores lógicos; literales numéricos, cadenas y valores booleanos.
Transformación y análisis	Procesamiento descriptivo y agregación de datos.	Agrupar por una o varias variables; aplicar conteo, suma, media, mediana, mínimo, máximo y desviación estándar; generar resúmenes de variables numéricas y categóricas. La correlación y la regresión lineal simple serán opcionales.

Componente	Capacidades del DSL	Funcionalidad mínima esperada
Visualización	Generación de representaciones gráficas a partir de los datos procesados.	Producir gráficas de barras, líneas, histogramas, dispersión y caja; configurar título, ejes y leyenda; mostrar la gráfica o exportarla en formato PNG.
Abstracción y re-utilización	Encapsulación de flujos frecuentes de transformación y análisis.	Definir y ejecutar funciones sencillas; recibir parámetros; retornar resultados; utilizar condicionales básicos para controlar o validar la ejecución.
Manejo de símbolos y tipos	Administración de variables, conjuntos de datos y resultados durante la ejecución.	Mantener una tabla de símbolos o contexto de ejecución; validar identificadores, columnas, tipos y operaciones permitidas; evitar el uso de variables no declaradas.
Errores y diagnóstico	Detección y comunicación de problemas en las diferentes etapas del procesamiento.	Reportar errores léxicos, sintácticos y semánticos con mensajes comprensibles; incluir, cuando sea posible, número de línea y columna; controlar archivos inexistentes y operaciones inválidas.
Interfaz de ejecución	Mecanismo mediante el cual el usuario ejecuta programas escritos en el DSL.	Ejecutar archivos desde una interfaz de línea de comandos. Una consola interactiva, interfaz gráfica o extensión para editor será opcional.

3. Enfoque de diseño

El DSL deberá privilegiar una sintaxis declarativa o funcional. Esto significa que el usuario describirá **qué transformación desea realizar**, mientras que el intérprete resolverá cómo ejecutarla mediante las bibliotecas de Python.

Principio de diseño

El DSL no debe convertirse en una copia completa de Python. Su valor está en ofrecer instrucciones compactas, coherentes y cercanas al vocabulario de la ciencia de datos.

Una alternativa recomendada es trabajar mediante encadenamiento de operaciones o *pipelines*. Cada operación recibe un conjunto de datos y produce uno nuevo, evitando modificar silenciosamente el resultado anterior.

```

1 # Cargar datos de ventas
2 ventas = cargar "datos/ventas.csv"
3
4 # Preparar el conjunto de datos
5 ventas_limpias = ventas
6     |> seleccionar [fecha, ciudad, categoria, unidades, precio]
7     |> filtrar donde unidades > 0 y precio > 0
8     |> crear total = unidades * precio
9

```

```

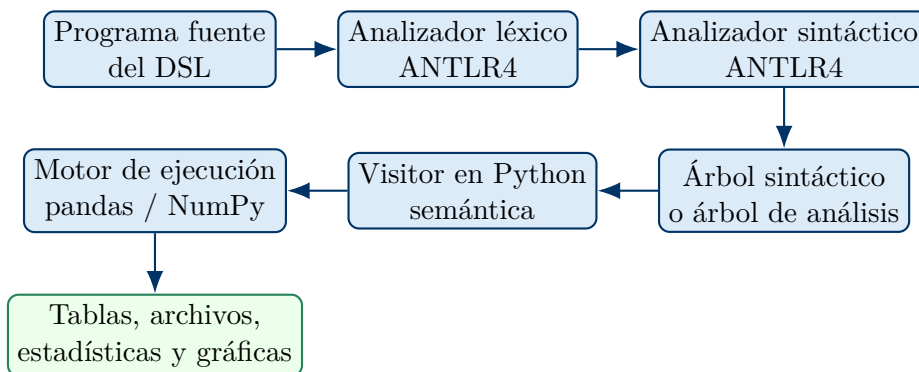
10 # Obtener un resumen por ciudad
11 resumen = ventas_limpias
12     |> agrupar por [ciudad]
13     |> resumir ingreso = suma(total),
14             promedio = media(total),
15             registros = contar()
16
17 # Crear y guardar una visualizacion
18 graficar barras resumen
19     x ciudad
20     y ingreso
21     titulo "Ingresos por ciudad"
22     guardar como "salidas/ingresos_ciudad.png"
23
24 # Exportar la tabla obtenida
25 guardar resumen como "salidas/resumen_ciudades.csv"

```

Listing 1: Ejemplo ilustrativo de un programa en el DSL

La sintaxis definitiva deberá ser propuesta y justificada por cada equipo. El ejemplo anterior funciona únicamente como referencia de diseño.

4. Arquitectura esperada



La solución deberá mantener módulos separados para:

- gramática léxica y sintáctica;
- generación y manejo del árbol;
- comprobaciones semánticas;
- operaciones sobre conjuntos de datos;
- generación de visualizaciones;
- interfaz de ejecución;
- pruebas y ejemplos.

5. Organización del proyecto por fases

El proyecto se desarrollará en tres fases acumulativas, correspondientes a los tres cortes del semestre. En cada corte se deberá entregar una versión ejecutable y demostrable; por tanto, la integración de los componentes no deberá aplazarse hasta la entrega final.

Corte	Fase	Propósito y actividades principales	Alcance mínimo de la versión	Entregables y producto verificable
Primero	Especificación y front-end del lenguaje	Delimitar el dominio y los casos de uso; definir usuarios, entradas, salidas y restricciones; diseñar palabras reservadas, operadores, literales y sentencias; elaborar la gramática BNF/EBNF; implementar la gramática en ANTLR4; generar lexer y parser para Python; validar programas correctos e incorrectos.	Asignaciones y expresiones básicas; carga de CSV; selección de columnas; filtros con comparaciones sencillas; reconocimiento sintáctico de una instrucción de visualización, aunque todavía no produzca la gráfica.	Documento de alcance; catálogo de instrucciones; gramática BNF/EBNF; archivo <code>.g4</code> ; código generado por ANTLR; pruebas léxicas y sintácticas; ejemplos; repositorio con <code>README.md</code> ; sustentación. Producto: el sistema reconoce programas del DSL, genera el árbol de análisis y reporta errores comprensibles.
Segundo	Semántica y procesamiento de datos	Implementar el patrón Visitor en Python; diseñar el contexto de ejecución y la tabla de símbolos; asociar identificadores con objetos de datos; implementar carga, selección, filtrado, ordenamiento, columnas calculadas y tratamiento de datos faltantes; incorporar agrupamiento, agregaciones, validaciones semánticas y pruebas.	Lectura y almacenamiento de datos tabulares; selección, filtrado y creación de columnas; agrupamiento y agregaciones; estadísticas descriptivas; escritura de resultados en CSV; errores por variables o columnas inexistentes, tipos incompatibles y operaciones no permitidas.	Gramática actualizada; Visitor y motor de procesamiento; tabla de símbolos; documentación de reglas semánticas; pruebas unitarias y de integración; mínimo tres programas completos; evidencia de transformación y exportación; repositorio actualizado; sustentación. Producto: el DSL procesa datos reales y produce una tabla resumen correctamente exportada.

Corte	Fase	Propósito y actividades principales	Alcance mínimo de la versión	Entregables y producto verificable
Tercero	Visualización, integración y producto final	Implementar instrucciones de visualización; incorporar títulos, ejes, leyendas y archivos de salida; integrar una interfaz de línea de comandos; mejorar la presentación de errores; validar un caso de estudio completo; estabilizar la arquitectura; preparar documentación, demostración y entrega final.	Al menos cuatro tipos de gráficas; configuración básica desde el DSL; exportación a PNG; ejecución completa desde consola; caso aplicado con datos reales o públicos; pruebas de regresión para conservar las funciones de las fases anteriores.	Código fuente completo; gramática final; intérprete o traductor ejecutable; manual de usuario; manual técnico; conjunto de pruebas y reporte; caso de estudio con datos, programa DSL, tablas y gráficas; demostración; repositorio final reproducible. Producto: un usuario ejecuta un programa para cargar, transformar, analizar, visualizar y guardar resultados sin modificar el código interno de Python.

6. Criterios de evaluación

La evaluación de cada corte puede considerar los siguientes criterios. Los porcentajes pueden ajustarse de acuerdo con la planeación del curso.

Criterio	Peso sugerido	Evidencia
Diseño del lenguaje y coherencia de la sintaxis	20 %	Gramática, ejemplos y decisiones justificadas
Correctitud léxica, sintáctica y semántica	25 %	Pruebas positivas y negativas
Implementación y calidad del código	20 %	Modularidad, legibilidad y uso adecuado del Visitor
Funcionalidad del dominio	20 %	Operaciones de datos y visualización
Documentación, reproducibilidad y uso de Git	10 %	Manuales, historial y ejecución limpia
Sustentación y demostración	5 %	Presentación y respuesta a preguntas

7. Requisitos técnicos

- ANTLR4 para la definición del lexer y el parser.
- Python 3.11 o superior como lenguaje de implementación recomendado.
- Patrón Visitor para ejecutar o traducir las construcciones del lenguaje.
- Bibliotecas sugeridas: `pandas`, `NumPy` y `Matplotlib`.
- Gestión del código mediante Git y un repositorio remoto.
- Archivo de dependencias mediante `requirements.txt` o `pyproject.toml`.
- Ejecución por línea de comandos. Una interfaz gráfica o plugin de editor será opcional.
- Datos de ejemplo incluidos o acompañados por instrucciones claras para obtenerlos.

8. Caso de estudio final recomendado

Cada equipo deberá seleccionar un conjunto de datos que permita demostrar el flujo completo del lenguaje. Algunas opciones son ventas, movilidad, datos ambientales, educación, salud pública o telecomunicaciones. El caso debe incluir:

1. descripción del conjunto de datos;
2. preguntas de análisis;
3. programa escrito en el DSL;
4. limpieza y transformación;
5. indicadores o estadísticas;
6. mínimo cuatro visualizaciones;

7. interpretación de resultados;
8. archivos generados y procedimiento de reproducción.

9. Resultado esperado del curso

Al finalizar el semestre, cada equipo deberá contar con un prototipo funcional que evidencie la relación entre teoría de lenguajes, gramáticas, análisis léxico, análisis sintáctico, semántica y traducción. El proyecto no se evaluará únicamente por la cantidad de instrucciones implementadas, sino por la coherencia del lenguaje, la claridad de su gramática, la calidad del procesamiento semántico y la reproducibilidad del producto.

Meta final: convertir un programa breve y legible del DSL en un flujo reproducible de ciencia de datos que produzca tablas, estadísticas y visualizaciones útiles.